DFSha: sistema de archivos distribuido

Diseño, implementación y validación grupal

Proyecto 1, Fase II

Andrés Felipe Martínez Velasquez

Juan José García Duque

Samuel Parra Cadavid

Universidad Pontificia Bolivariana

Arquitecturas de Nube y Sistemas Distribuidos

Álvaro Ospina Sanjuan

6 de noviembre de 2026

1. Introducción y alcance

DFSha es un sistema de archivos distribuido cliente/servidor con comunicación entre servidores (S2S), correspondiente a la opción 1 del proyecto. Separa la coordinación de operaciones del almacenamiento de bloques, siguiendo principios utilizados en GFS y HDFS [1], [2]. El cliente administra archivos y directorios dentro del espacio privado de cada usuario, sin conocer su ubicación física.

Esta fase integra las tres propuestas individuales y presenta la arquitectura implementada, sus protocolos, las garantías de concurrencia y las pruebas aportadas por el equipo. El prototipo se desplegó en cuatro máquinas virtuales EC2 de AWS Academy. Allí se registraron pruebas funcionales, caídas de servicios y mediciones de rendimiento. Las pruebas de fallo detuvieron contenedores dentro de las máquinas; no simularon la pérdida completa de una instancia EC2 ni de su disco.

2. Comparación de las propuestas individuales

Tabla 1. Comparación y selección arquitectónica

Criterio	Andrés Martínez	Juan José García	Samuel Parra	Solución grupal
Control	Dos APIs y tres etcd/Raft	PostgreSQL con propuesta activo/pasivo	Tres coordinadores y tres etcd/Raft	Dos APIs equivalentes y tres etcd/Raft
Bloque	8 MiB	4 MiB	4 MiB	8 MiB; último bloque menor
Replicación	R=3, W=2; S2S	R=2; confirman ambas	R=3, W=2; S2S	R=3 objetivo; W=2 verificado
Concurrencia	Lock por bloque y fencing	Lock por bloque y versión esperada	Lock por archivo y CAS	rw por bloque; w/a exclusivos
Lectores	Versión fijada y gracia	Manifiesto estable	Sesiones renovables	Snapshot con lease y retención por referencias
Seguridad	Argon2id, RS256 y mTLS	Argon2id, RS256 y mTLS	Argon2id, TLS/mTLS y AES-GCM	scrypt, firmas EdDSA, mTLS y AES-GCM

La topología de Andrés se adoptó como base porque especifica dos servicios de metadatos sin estado y un catálogo con consenso, sin depender de un controlador único. De García se incorpora la validación de la versión esperada del bloque; de Samuel, las sesiones lectoras renovables que permiten conservar versiones mientras se utilizan. La réplica S2S y la lectura por versión son decisiones coincidentes entre propuestas.

La solución PostgreSQL activo/pasivo de García es una alternativa válida, pero su propuesta dejaba el failover sujeto al tiempo disponible y contemplaba replicación asíncrona. Para el prototipo se prefirió el catálogo etcd/Raft con leases y transacciones [3]. El lock por archivo de Samuel se reservó para truncar o anexar; la modificación de bloques distintos conserva

paralelismo. Se mantuvieron bloques de 8 MiB, aunque las pruebas disponibles no comparan todavía su rendimiento con 4 MiB y no permiten afirmar que sea el tamaño óptimo.

3. Decisiones arquitectónicas y límites

Tabla 2. Decisiones implementadas y sus consecuencias

Decisión	Justificación y compromiso
Separación de planos	Las APIs organizan permisos, rutas y versiones; los bytes viajan directamente entre cliente y nodos.
Preparación con W=2	Se exigen recibos firmados de dos nodos distintos antes de publicar; una tercera réplica puede quedar pendiente.
Bloques inmutables	Cada modificación crea un identificador nuevo. Un reintento idéntico es idempotente; no reemplaza contenido publicado.
Locks y versión esperada	rw bloquea los bloques afectados y rechaza bases viejas. w y a requieren exclusión entre escritores del archivo.
Snapshots renovables	Un lease mantiene las referencias de lectura; el GC no depende exclusivamente de un tiempo fijo de gracia.
Commit acotado	Las fichas se preparan por bloque; la publicación compara sesión y epoch sin añadir una operación transaccional por bloque.
Resultados idempotentes	Repetir el mismo cierre devuelve la versión ya publicada; otro cuerpo para la misma operación recibe conflicto.
Identidades separadas	Firmantes de usuario, capacidades y recibos distintos; CA de catálogo aislada y KEK solo en coordinadores.

El epoch global simplifica la coordinación de referencias y del namespace, pero introduce contención entre operaciones de metadatos. No se promete escalabilidad ilimitada. El límite configurado es 8192 bloques por archivo, equivalentes a 64 GiB con bloques de 8 MiB. Una adquisición atómica de lock admite hasta 48 bloques y un rename atómico, hasta 48 entradas. El formato de catálogo de esta revisión no migra automáticamente los datos de versiones previas.

La replicación S2S se realiza secuencialmente desde un bloque validado. El nodo receptor verifica el contenido, lo persiste y después lo transmite al siguiente destino. No se declara un pipeline solapado. Tampoco se presupone semántica POSIX completa, compartición entre usuarios o cuotas.

4. Componentes, metadatos y despliegue

El plano de control tiene dos APIs FastAPI equivalentes. Ambas usan tres miembros etcd replicados mediante Raft [9], con cuórum de dos de tres. El catálogo conserva usuarios, rutas, inodos, versiones, sesiones, locks, planes de bloque, réplicas y resultados de operaciones. El plano de datos tiene cuatro nodos que almacenan bloques cifrados en volúmenes persistentes.

DFSha · componentes de la revisión

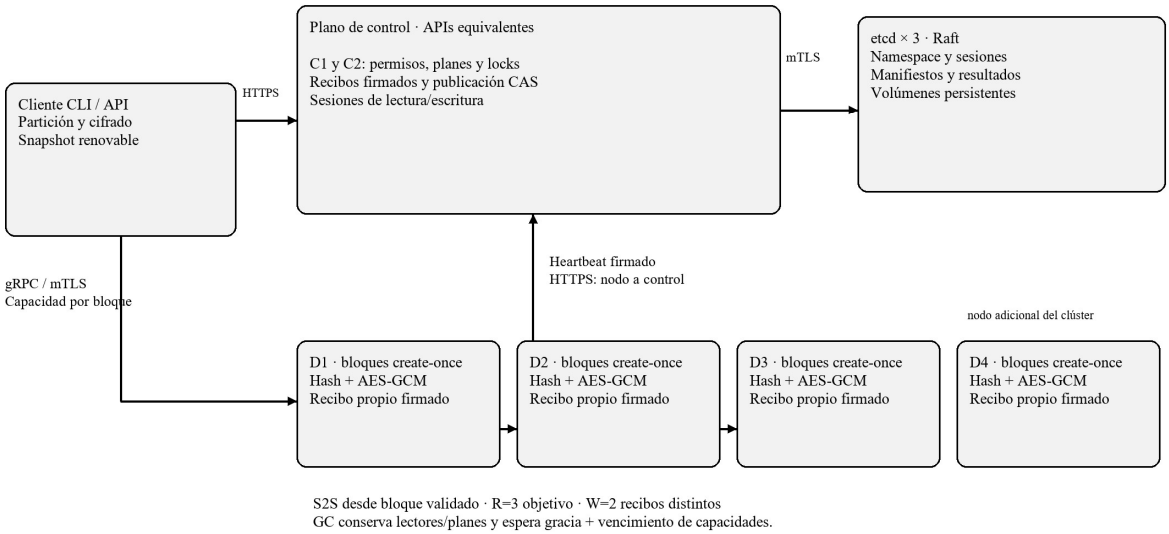
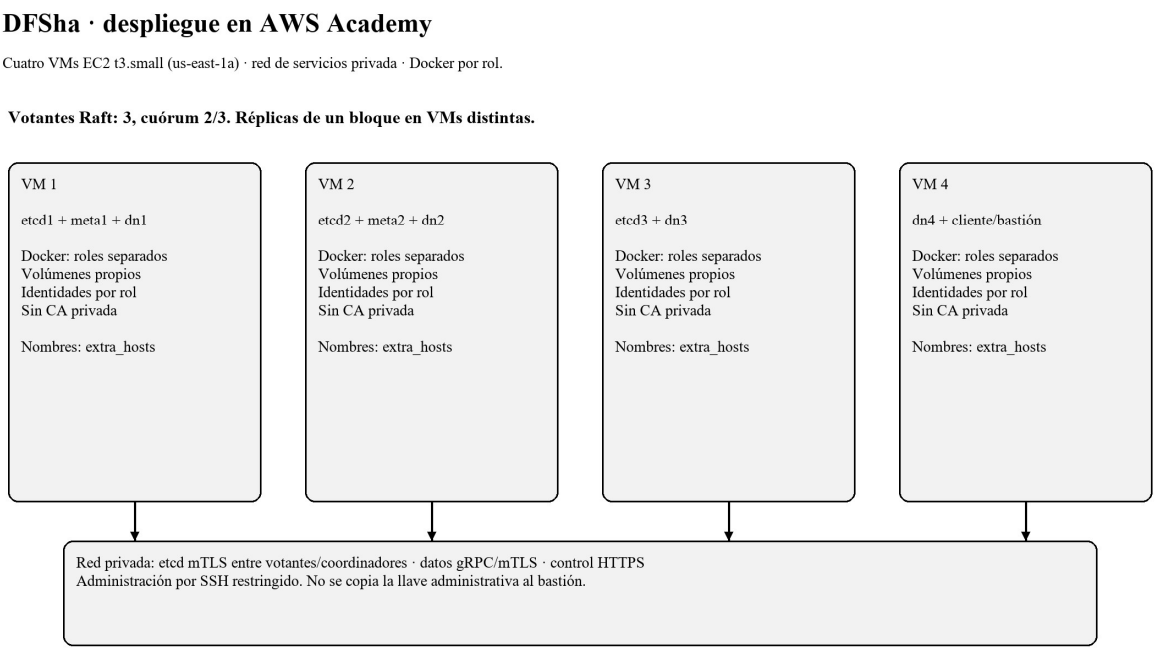


Tabla 3. Distribución de servicios en AWS Academy

Máquina	Servicios	Persistencia
VM1	etcd1, coordinador meta1 y nodo dn1	Volúmenes separados de catálogo y bloques
VM2	etcd2, coordinador meta2 y nodo dn2	Volúmenes separados de catálogo y bloques
VM3	etcd3 y nodo dn3	Volúmenes separados de catálogo y bloques
VM4	Nodo dn4 y cliente/bastión	Volumen de bloques y espacio de trabajo del cliente

Las cuatro máquinas usan Ubuntu 24.04 y contenedores Docker con red del anfitrión. Los nombres de servicio se resuelven con entradas de hosts configuradas también dentro de cada contenedor; no se presenta este mecanismo como un DNS con comprobaciones de salud. El cliente recibe las ubicaciones actuales de sus bloques en el manifiesto. Cada réplica de un bloque reside en una máquina distinta, al igual que los tres votantes de etcd.



5. Requisitos funcionales, protocolos y APIs

5.1. Operaciones del servicio

RF1. Gestión jerárquica. ls lista entradas; cd mantiene la ruta actual del cliente y valida el directorio; mkdir crea directorios; rmdir elimina solo directorios vacíos y rm retira archivos. rename permite mover o renombrar entradas. Cada usuario autenticado trabaja en su propio espacio lógico.

RF2. Transferencia completa. put, equivalente a send, divide, cifra y distribuye un archivo completo. get, equivalente a receive/recv, recupera sus bloques en orden y comprueba su integridad antes de reemplazar el destino local.

RF3. Acceso a archivos. open fija una versión; read consulta rangos de esa versión; lock reserva bloques; write admite creación, reemplazo, modificación parcial y anexado según el modo; close publica las escrituras pendientes o libera una lectura. La inmutabilidad se aplica a cada identificador de bloque, no impide modificar el archivo lógico.

Tabla 4. Modos de apertura y escritura

Modo	Semántica y concurrencia
r	Lectura de un snapshot con lease renovable; no admite write.
w	Reemplazo desde cero, incluido archivo vacío; excluye otros escritores. Con create permite crear el archivo.
rw	Modificación parcial por bloques. Rechaza cambios sobre una base que otro escritor ya modificó.
a	Anexa desde EOF y puede aumentar el tamaño. Excluye otros escritores para coordinar la posición final.

Una escritura por posición requiere offset menor o igual al tamaño actual: no se permiten huecos más allá de EOF. Los cambios generan bloques nuevos mediante copia sobre escritura. La autenticación y autorización protegen RF1, RF2 y RF3.

5.2. API del plano de control

El control usa REST sobre HTTPS y cuerpos JSON. Las operaciones autenticadas llevan un JWT en Authorization. Los siguientes endpoints describen el contrato implementado; los códigos de error permiten distinguir sesión inválida, recurso bloqueado, conflicto e indisponibilidad.

Tabla 5. Operaciones REST

Método y ruta	Función y respuesta principal
POST /v1/auth/login	Autenticar. 200 con token; 401 con credenciales inválidas.
GET /v1/fs/ls?path=...	Listar entradas autorizadas. 200; 404 si no existe.
POST /v1/fs/mkdir	Crear directorio. 201; 409 si existe.
DELETE /v1/fs/rmdir?path=...	Eliminar directorio vacío. 204; 409 si contiene entradas.
DELETE /v1/fs/rm?path=...	Retirar archivo. 204; 423 si hay escritores activos.
POST /v1/fs/rename	Mover/renombrar. 204; 409 por destino existente, 423 por escritores activos.
POST /v1/files/open	Abrir y fijar versión. 200 con fd, versión, manifiesto, DEK y TTL.
POST /v1/files/{fd}/keepalive	Renovar lease. 200; 410 si venció.
GET /v1/files/{fd}/manifest	Refrescar destinos/capacidades del mismo snapshot. 200.
POST /v1/files/{fd}/lock	Reservar rango. 200; 423 si está ocupado, 409 con base vieja.
POST /v1/files/{fd}/unlock	Liberar un rango sin carga pendiente. 204; 409 si tiene plan pendiente.
POST /v1/files/{fd}/blocks	Reservar ID, destinos y fencing. 200; 503 sin dos nodos disponibles.
POST /v1/files/{fd}/blocks/{bid}/authorize	Fijar hash/tamaño y emitir capacidad. 200; 409 si cambia un plan ya fijado.
POST /v1/files/{fd}/blocks/{bid}/prepared	Verificar recibos. 200; 409 BAD_RECEIPT si son inválidos o duplicados.

Método y ruta	Función y respuesta principal
POST /v1/files/{fd}/close	Publicar/recuperar resultado del cierre. 200; 409 por conflicto, 410 por sesión vencida.
POST /v1/files/{fd}/abort	Abortar la sesión y dejar residuos no publicados para GC. 204.
GET /v1/files/{fd}/status	Consultar operación abierta/cerrada. 200; 410 si no hay sesión o resultado.
GET /v1/files/stat?path=...	Consultar versión, tamaño y réplicas. 200; 404.
GET /v1/admin/nodes	Consultar estado del clúster. 200; 403 sin rol admin.

5.3. API del plano de datos

Los bloques viajan por gRPC con streaming y mTLS [4]. Una capacidad firmada autoriza operación, bloque, contenido y destinos. La escritura incluye identificador del bloque, identificador de operación, tamaño cifrado esperado, SHA-256 y cadena de destinos.

Tabla 6. Operaciones gRPC

RPC	Emisor y efecto
WriteBlock	Cliente o nodo anterior. Recibe fragmentos, valida, hace fsync, crea el bloque sin sobrescribir y devuelve recibos firmados.
ReadBlock	Cliente. Entrega fragmentos cifrados; el cliente valida SHA-256 y la etiqueta GCM.
ReplicateBlock	Coordinador. Ordena una copia S2S con tamaño y hash del catálogo; una fuente corrupta se rechaza.
DeleteBlock	Recolector autorizado. Elimina una copia de un bloque sin referencias válidas.
ListBlocks	Servicio autorizado. Obtiene inventario de bloques según el contrato de mantenimiento.

5.4. Ejemplo de solicitud, éxito y error

El siguiente intercambio reserva el bloque 0 de una sesión ya abierta. Los identificadores y firmas son ilustrativos; los recibos reales se generan con la clave de cada nodo.

Solicitud de reserva. El cliente conserva el request_id si repite la misma petición.

```
POST /v1/files/{fd}/blocks
{"index": 0, "request_id": "op-b0"}
```

Respuesta exitosa. El coordinador asigna el bloque y los nodos autorizados.

```
200 OK
{
  "block_id": "0123456789abcdef0123456789abcdef",
  "fencing": 1234,
  "w": 2,
  "pipeline": ["dn1@dn1.dfsha.internal:50051",
               "dn2@dn2.dfsha.internal:50051",
               "dn3@dn3.dfsha.internal:50051"]
}
```

Preparación y publicación. El cliente cifra el bloque, fija su hash y tamaño mediante authorize y lo transfiere por WriteBlock. Después remite los recibos a prepared. El coordinador comprueba al menos dos firmantes distintos antes de permitir el cierre.

```
POST /v1/files/{fd}/blocks/{bid}/prepared
{"receipts": ["firma_dn1", "firma_dn2"]}
```

Respuesta de error. Un recibo duplicado, falso o ajeno al plan no prepara el bloque.

```
409 Conflict
{
  "detail": {
    "code": "BAD_RECEIPT",
    "msg": "confirmación falsa, duplicada o fuera del plan"
  }
}
```

close solo publica los bloques de los planes vigentes y preparados. Repetir el mismo fd y cuerpo devuelve la versión ya publicada; un cuerpo diferente para la operación cerrada recibe conflicto. Un 409 RETRY del catálogo se reintenta con espera aleatoria acotada. Un 409 VERSION_CONFLICT exige reabrir o releer y no se oculta con reintentos.

6. Consistencia, concurrencia y tolerancia a fallos

6.1. Publicación y lectores

Los metadatos se coordinan mediante transacciones de etcd [3], [9]. Para publicar, el servidor comprueba los planes preparados, los locks, la sesión y los tamaños del manifiesto. La nueva versión aparece mediante una transacción CAS; antes del éxito no es visible. Las nuevas aperturas observan la última versión confirmada. Los lectores que abrieron antes conservan su snapshot, según la visibilidad close-to-open [5].

Las sesiones tienen un lease de 60 s, renovado cada 10 s. Una sesión lectora vigente mantiene referencias a sus bloques y permite renovar capacidades sin cambiar la versión. Al cerrar o expirar, deja de retener los bloques obsoletos. El GC marca huérfanos, espera gracia y vencimiento de capacidades, y revalida referencias antes de borrar. El plazo de gracia no sustituye el seguimiento de lectores [11].

6.2. Escrituras concurrentes

En rw, adquirir el lock también verifica el identificador del bloque base. Otro escritor que lo haya modificado desde open provoca 409 VERSION_CONFLICT. Dos clientes que afectan bloques distintos pueden preparar sus cambios en paralelo y el cierre conserva ambas modificaciones. w y a excluyen otros escritores para coordinar truncado y EOF. rm y rename responden 423 mientras haya escritores activos.

Todos los locks de una sesión comparten su lease. Si el cliente desaparece, vencen junto con la sesión; un cierre posterior no puede publicar. La renovación continúa mientras close transfiere

pendientes. El resultado del cierre queda asociado a la operación para que una respuesta perdida no produzca una publicación duplicada.

6.3. Detección, reparación y disponibilidad

Los nodos envían heartbeats firmados cada 3 s. Sin latido durante 10 s pasan a sospechosos y dejan de recibir nuevas asignaciones; a los 30 s se consideran caídos. El monitor renueva su liderazgo en paralelo y repara desde una fuente cuyo hash/tamaño coinciden con el catálogo. El destino se registra antes del RPC para que el GC conozca posibles copias que terminen tarde. Una fuente corrupta no se anuncia como reparación válida.

Con dos miembros etcd comunicados se mantiene el cuórum. Sin mayoría, las operaciones de catálogo fallan cerrado con 503; no se publica una versión sobre un catálogo inconsistente [10]. Una lectura ya abierta puede utilizar réplicas disponibles dentro de la vigencia de su sesión y capacidades. No se garantiza operación ante cualquier combinación de fallos o pérdida de todas las copias de un bloque.

7. Seguridad e integridad

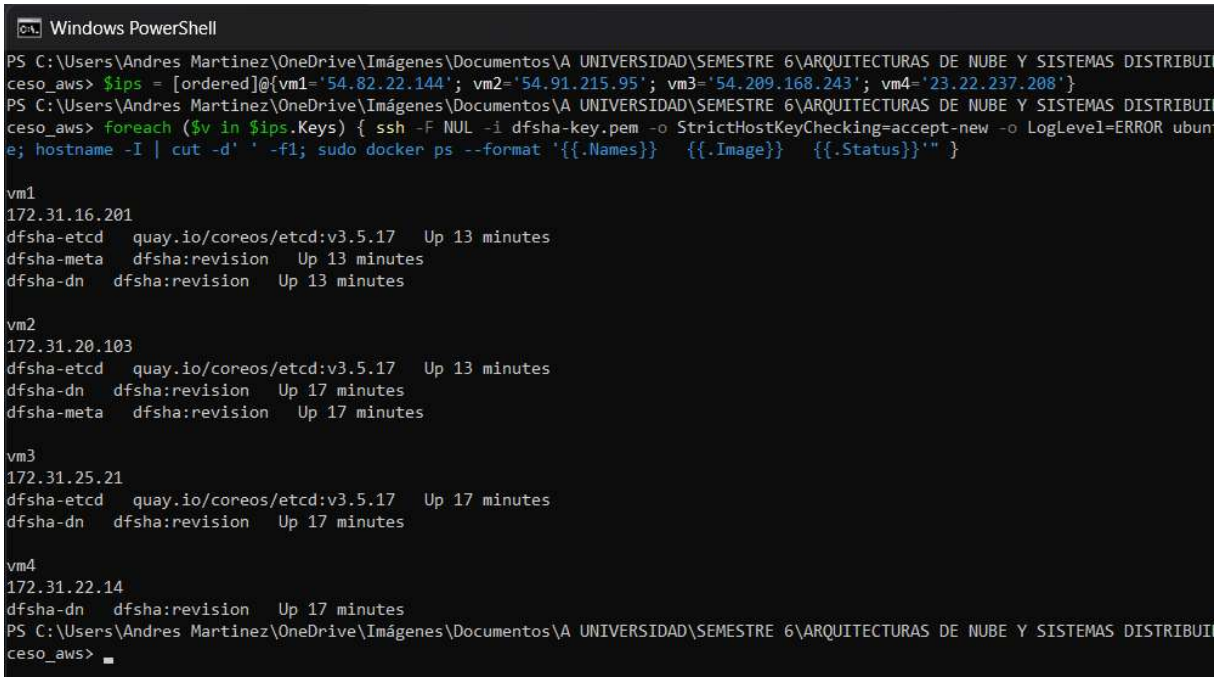
Cada servicio tiene una identidad de transporte propia. La CA de etcd habilita a los miembros del catálogo y coordinadores; nodos y clientes no reciben sus credenciales ni la KEK. gRPC exige certificado de cliente y capacidad de bloque. JWT de usuario y capacidades tienen claves EdDSA separadas; cada nodo firma sus recibos con Ed25519. Las contraseñas usan scrypt con sal y parámetros N=32768, r=8, p=3. Los JWT duran 15 minutos [6].

TLS protege las comunicaciones [7]. En reposo, el cliente cifra los bloques con AES-256-GCM y AAD igual al block_id [8]. La DEK de cada archivo está envuelta con la KEK disponible en los coordinadores. El modelo confía en el coordinador para autorizar y entregar la DEK; no declara cifrado extremo a extremo resistente al servidor.

Se comprueban tamaño y SHA-256 del bloque cifrado, etiqueta GCM y hash canónico del manifiesto. put/w registra además el SHA-256 de todos los bytes planos; rw/a lo deja en null porque no lo recalcula. El hash de manifiesto no se confunde con el hash plano global. get escribe un temporal y reemplaza el archivo de destino solo después de validar la longitud y la integridad.

8. Validación en AWS Academy

Los registros aportados por el equipo corresponden a cuatro VMs EC2 t3.small en us-east-1a. La Figura 3 muestra los contenedores por VM y sus direcciones privadas. La evidencia funcional posterior incluye la consulta de los cuatro nodos de datos y la distribución de bloques. Las capturas se conservan junto a los resultados de la misma etapa de validación.



La regresión final del 6 de noviembre de 2026 reporta 19 casos, 0 fallos, 0 errores y 0 omitidos. Comparte huella de código con el benchmark completo seleccionado. Incluye recibos falsos/duplicados, inmutabilidad, snapshot sin caché, base vieja, modificación de bloques distintos, vacío, rm/rename, cierre idempotente, rangos, aislamiento, mTLS y clientes concurrentes. La prueba grande transfiere y recupera 129 bloques de 8 MiB, equivalentes a 1032 MiB, con hash coincidente.

```
ubuntu@vm4: ~/dfsha
(.venv) ubuntu@vm4:~/dfsha$ RUN_LARGE=1 python -m tests.regression 2>&1 | tail -24
test_block_replay_cannot_change_contents (__main__.Regression.test_block_replay_cannot_change_contents) ... ok
test_cli_failed_write_releases_session (__main__.Regression.test_cli_failed_write_releases_session) ... ERROR 423 BLOCK_LOCK
oque
ok
test_client_has_no_catalog_or_signing_secrets (__main__.Regression.test_client_has_no_catalog_or_signing_secrets) ... ok
test_commit_retry_is_idempotent (__main__.Regression.test_commit_retry_is_idempotent) ... ok
test_concurrent_clients_survive_epoch_conflicts (__main__.Regression.test_concurrent_clients_survive_epoch_conflicts) ... ok
test_distinct_blocks_preserve_both_changes (__main__.Regression.test_distinct_blocks_preserve_both_changes) ... ok
test_empty_replacement (__main__.Regression.test_empty_replacement) ... ok
test_forged_and_duplicate_receipts_rejected (__main__.Regression.test_forged_and_duplicate_receipts_rejected) ... ok
test_invalid_ranges_and_r1 (__main__.Regression.test_invalid_ranges_and_r1) ... ok
test_large_file_beyond_128_blocks (__main__.Regression.test_large_file_beyond_128_blocks) ... ok
test_mtls_rejects_missing_client_certificate (__main__.Regression.test_mtls_rejects_missing_client_certificate) ... ok
test_partial_stream_never_prepares_a_block (__main__.Regression.test_partial_stream_never_prepares_a_block) ... ok
test_partial_write_has_manifest_hash (__main__.Regression.test_partial_write_has_manifest_hash) ... ok
test_rm_and_rename_reject_active_writer (__main__.Regression.test_rm_and_rename_reject_active_writer) ... ok
test_roundtrip_and_snapshot_without_cache (__main__.Regression.test_roundtrip_and_snapshot_without_cache) ... ok
test_stale_block_is_rejected (__main__.Regression.test_stale_block_is_rejected) ... ok
test_unprepared_commit_rejected (__main__.Regression.test_unprepared_commit_rejected) ... ok
test_user_isolation (__main__.Regression.test_user_isolation) ... ok

----------------------------------------------------------------------
Ran 19 tests in 67.380s

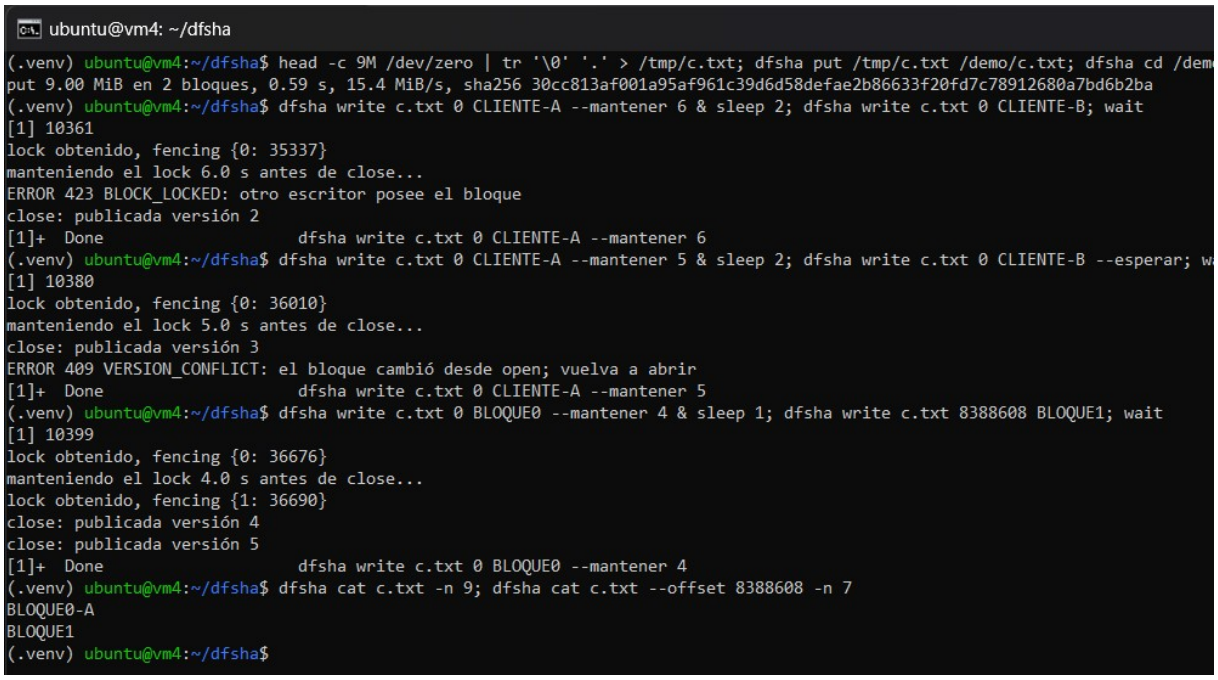
OK
(.venv) ubuntu@vm4:~/dfsha$
```

La demostración funcional carga, consulta y descarga un archivo de 200 MiB con SHA-256 coincidente; también modifica, anexa y lee contenido desde la CLI. La consulta de estado identifica los nodos donde se almacenan las copias.

```
ubuntu@vm4: ~/dfsha
(.venv) ubuntu@vm4:~/dfsha$ dfsha login admin admin123; dfsha nodes
sesión iniciada como admin (token válido 15 min)
atendido por meta1, monitor líder meta1, bloques 1515, subreplicados 0
  dn1  vivo       dn1.dfsha.internal:50051     bloques 1104  último latido hace 1.0 s
  dn2  vivo       dn2.dfsha.internal:50051     bloques 1162  último latido hace 0.1 s
  dn3  vivo       dn3.dfsha.internal:50051     bloques 1163  último latido hace 3.0 s
  dn4  vivo       dn4.dfsha.internal:50051     bloques 1134  último latido hace 0.1 s
(.venv) ubuntu@vm4:~/dfsha$ dfsha login andres clave123; dfsha mkdir /demo; head -c 200M /dev/urandom > /tmp/grande.bin; sha
sesión iniciada como andres (token válido 15 min)
e6eca42c7858ce6556e34b637e1c96b8f0f041b2e11001823e776b798b5bfa3f  /tmp/grande.bin
(.venv) ubuntu@vm4:~/dfsha$ dfsha put /tmp/grande.bin /demo/grande.bin
put 200.00 MiB en 25 bloques, 5.87 s, 34.0 MiB/s, sha256 e6eca42c7858ce6556e34b637e1c96b8f0f041b2e11001823e776b798b5bfa3f
(.venv) ubuntu@vm4:~/dfsha$ dfsha stat /demo/grande.bin | head -6
/demo/grande.bin  versión 1  200.00 MiB  sha256 e6eca42c7858ce6556e34b637e1c96b8f0f041b2e11001823e776b798b5bfa3f
  bloque   0  20200d0af259  8.00 MiB  réplicas ['dn1', 'dn2', 'dn3']  vivas ['dn1', 'dn2', 'dn3']
  bloque   1  4d1c30434312  8.00 MiB  réplicas ['dn1', 'dn3', 'dn4']  vivas ['dn3', 'dn4', 'dn1']
  bloque   2  a78f6d1cfbc1  8.00 MiB  réplicas ['dn1', 'dn2', 'dn4']  vivas ['dn4', 'dn1', 'dn2']
  bloque   3  a9c8e8f824ed  8.00 MiB  réplicas ['dn2', 'dn3', 'dn4']  vivas ['dn2', 'dn3', 'dn4']
  bloque   4  ed691be01b65  8.00 MiB  réplicas ['dn1', 'dn2', 'dn3']  vivas ['dn2', 'dn3', 'dn1']
(.venv) ubuntu@vm4:~/dfsha$ dfsha get /demo/grande.bin /tmp/rec.bin; sha256sum /tmp/rec.bin
get 200.00 MiB (versión 1) en 2.34 s, 85.4 MiB/s, sha256 e6eca42c7858ce6556e34b637e1c96b8f0f041b2e11001823e776b798b5bfa3f ve
e6eca42c7858ce6556e34b637e1c96b8f0f041b2e11001823e776b798b5bfa3f  /tmp/rec.bin
(.venv) ubuntu@vm4:~/dfsha$ dfsha write /demo/notas.txt 0 'hola mundo'; dfsha append /demo/notas.txt ' desde AWS'; dfsha cat
lock obtenido, fencing {0: 34070}
close: publicada versión 1
lock obtenido, fencing {0: 34111}
close: publicada versión 2
hola mundo desde AWS
(.venv) ubuntu@vm4:~/dfsha$
```

El segundo escritor sobre el mismo bloque recibe 423 BLOCK_LOCKED. Si espera el lock y el primero publica, recibe 409 VERSION_CONFLICT porque la base observada en open ya

cambió. Debe reabrir/releer. En bloques distintos, las dos modificaciones quedan publicadas. Estas salidas corresponden a la política de locks y versión esperada.



8.1. Caídas de servicios y recuperación observada

Tabla 7. Resultado de las sondas durante los fallos

Condición	Observación registrada
dn1 detenido	La lectura y una carga nueva completan; las réplicas del archivo de prueba se restauran sin dn1.
meta1 detenido	La lectura continúa mediante meta2.
Un etcd detenido	La lectura se completa con dos votantes comunicados.
Dos etcd detenidos	El catálogo rechaza la operación con 503 por pérdida de cuórum.
Servicios reiniciados	La sonda de lectura posterior vuelve a completar.

El comando que detuvo dn1 finalizó a las 02:22:39.792 UTC. La primera sonda que confirmó tres copias sin dn1 terminó a las 02:23:21.187 UTC. El intervalo observado fue de 41.4 segundos. Incluye detección del fallo, reparación y espera entre sondeos; no mide exclusivamente la transferencia de réplicas ni fija el instante exacto de restauración, que pudo ocurrir entre consultas.

Se conservaron tres sondeos de reparación fallidos antes del exitoso. Todos los pasos no asociados a esa espera terminaron con código cero y el registro muestra el reinicio de los servicios y una lectura final correcta. Estas pruebas detienen procesos Docker en VMs reales; la caída completa de una VM con todos sus roles es un escenario distinto.

```
Windows PowerShell
PS C:\Users\Andres Martinez\OneDrive\Imágenes\Documentos\A UNIVERSIDAD\SEMESTRE 6\ARQUITECTURAS DE NUBE Y SISTEMAS DISTRIBUI
po_equipo\dfsha> (Get-Content resultados\revision-failures-aws-abe1be38f48a\run.json -Raw -Encoding UTF8 | ConvertFrom-Json)
tep, returncode, seconds, stdout -AutoSize

vm  step             returncode seconds stdout
--  ----             ---------- ------- ------
vm4 sonda seed                0    4,24 OK: seed
vm1 detener dn                0   13,03 dfsha-dn
vm4 sonda check               0    2,45 OK: check
vm4 sonda write               0    2,84 OK: write
vm4 sonda repaired            1    2,18
vm4 sonda repaired            1    2,27
vm4 sonda repaired            1    2,38
vm4 sonda repaired            0    2,26 OK: repaired
vm1 iniciar dn                0    2,05 dfsha-dn
vm1 detener meta              0    5,01 dfsha-meta
vm4 sonda check               0    2,48 OK: check
vm1 iniciar meta              0    2,13 dfsha-meta
vm1 detener etcd              0    2,87 dfsha-etcd
vm4 sonda check               0    2,59 OK: check
vm2 detener etcd              0    2,82 dfsha-etcd
vm4 sonda unavailable         0   11,88 OK: pérdida de cuórum rechazada con 503
vm2 iniciar etcd              0    1,77 dfsha-etcd
vm1 iniciar etcd              0    1,81 dfsha-etcd
vm4 sonda check               0    2,39 OK: check


PS C:\Users\Andres Martinez\OneDrive\Imágenes\Documentos\A UNIVERSIDAD\SEMESTRE 6\ARQUITECTURAS DE NUBE Y SISTEMAS DISTRIBUI
po_equipo\dfsha>
```

Las pruebas de seguridad muestran 401 sin sesión o con contraseña incorrecta, 404 para una ruta ajena y 403 al consultar administración sin ese rol. Un nodo rechaza tokens de bloque ausentes o alterados y conexiones sin TLS. Las capturas muestran negociación TLS 1.3; no demuestran por sí solas rechazo de todas las versiones anteriores.

```
ubuntu@vm4: ~/dfsha
(.venv) ubuntu@vm4:~/dfsha$ dfsha logout; dfsha ls /
ERROR 401 UNAUTHENTICATED: token ausente, inválido o vencido
(.venv) ubuntu@vm4:~/dfsha$ dfsha login andres claveMALA
ERROR 401 UNAUTHENTICATED: usuario o contraseña incorrectos
(.venv) ubuntu@vm4:~/dfsha$ dfsha login laura clave123; dfsha stat /demo/grande.bin; dfsha nodes
sesión iniciada como laura (token válido 15 min)
ERROR 404 NOT_FOUND: archivo inexistente
ERROR 403 FORBIDDEN: se requiere admin
(.venv) ubuntu@vm4:~/dfsha$ python -m tests.token_nodo /demo/grande.bin
dn1.dfsha.internal:50051 con token válido: OK, 8388636 bytes cifrados
dn1.dfsha.internal:50051 con sin token: UNAUTHENTICATED (capacidad inválida)
dn1.dfsha.internal:50051 con token alterado: UNAUTHENTICATED (capacidad inválida)
dn1.dfsha.internal:50051 sin TLS: UNAVAILABLE
(.venv) ubuntu@vm4:~/dfsha$ echo | openssl s_client -connect meta1.dfsha.internal:8443 -CAfile $CA_CERT 2>/dev/null | grep -
return"
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Verify return code: 0 (ok)
(.venv) ubuntu@vm4:~/dfsha$ echo | openssl s_client -connect dn1.dfsha.internal:50051 -alpn h2 -CAfile $CA_CERT -cert $CERTS
y 2>/dev/null | grep -E "Protocol|Cipher is|Verify return"
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Verify return code: 0 (ok)
(.venv) ubuntu@vm4:~/dfsha$
```

9. Resultados de rendimiento y análisis

El benchmark completo comenzó el 6 de noviembre de 2026 a las 02:29 UTC y finalizó a las 02:35 UTC. Contiene 80 muestras: 5 repeticiones en cada una de 16 condiciones de operación. Se registraron tamaños, concurrencia, replicación, dependencias y huella de código. Los resúmenes se obtienen de las muestras; el throughput es la media del tamaño dividido por la

duración de cada repetición. E1/E2 incluyen cifrado, hashes, recibos y publicación; E3 mide el tiempo de pared de los clientes concurrentes.

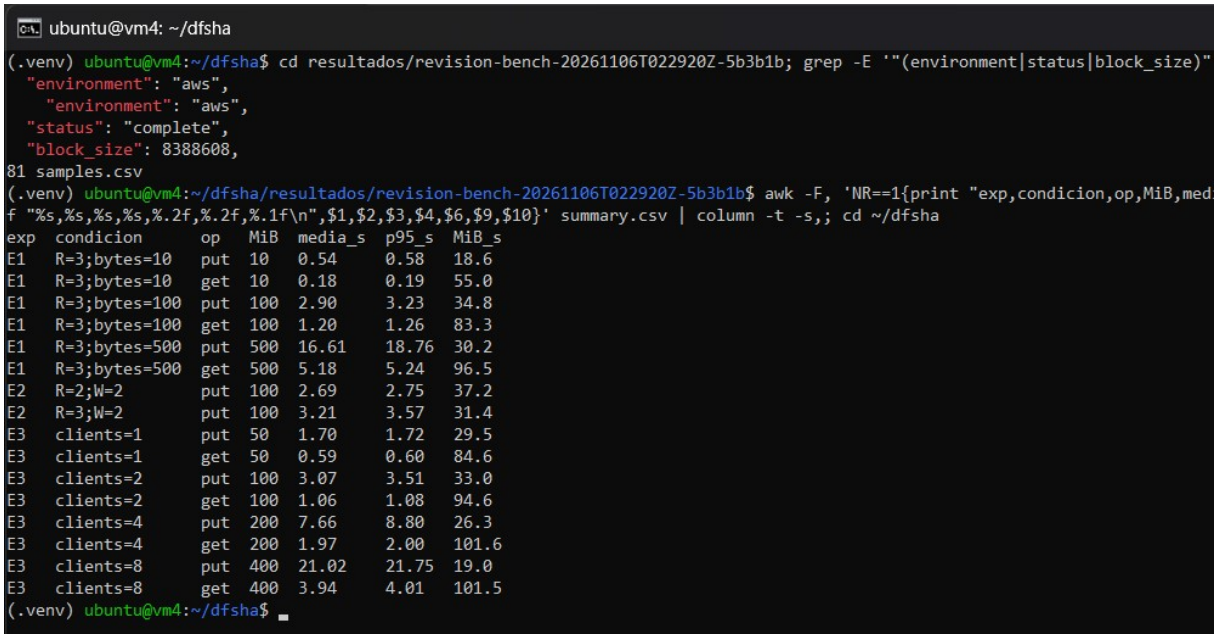
Tabla 8. Latencia y throughput medidos en AWS

Exp.	Condición	Op.	MiB	Media s	p95 s	MiB/s
E1	R3; 10	put	10	0.54	0.58	18.6
E1	R3; 10	get	10	0.18	0.19	55.0
E1	R3; 100	put	100	2.90	3.23	34.8
E1	R3; 100	get	100	1.20	1.26	83.3
E1	R3; 500	put	500	16.61	18.76	30.2
E1	R3; 500	get	500	5.18	5.24	96.5
E2	R=2/W2	put	100	2.69	2.75	37.2
E2	R=3/W2	put	100	3.21	3.57	31.4
E3	Clientes: 1	put	50	1.70	1.72	29.5
E3	Clientes: 1	get	50	0.59	0.60	84.6
E3	Clientes: 2	put	100	3.07	3.51	33.0
E3	Clientes: 2	get	100	1.06	1.08	94.6
E3	Clientes: 4	put	200	7.66	8.80	26.3
E3	Clientes: 4	get	200	1.97	2.00	101.6
E3	Clientes: 8	put	400	21.02	21.75	19.0
E3	Clientes: 8	get	400	3.94	4.01	101.5

E1: para 500 MiB, la escritura alcanzó 30.2 MiB/s y la lectura 96.5 MiB/s. La lectura no necesita producir nuevas réplicas. E2: con W=2, R=2 obtuvo 37.2 MiB/s y R=3 obtuvo 31.4 MiB/s, una reducción observada de 15.4 %. Esta comparación cuantifica la diferencia entre esas configuraciones; no permite deducir cuánto costaría otro algoritmo de replicación no medido.

E3: la escritura agregada pasó de 29.5 MiB/s con un cliente a 19.0 MiB/s con ocho; la lectura pasó de 84.6 a 101.5 MiB/s. La contención del epoch global es una explicación compatible con los conflictos RETRY y motivó reintentos con espera aleatoria. Como no se registraron contadores de reintentos ni series de CPU, disco y NIC, no se atribuye de forma concluyente a un recurso el costo observado.

La primera corrida AWS se conserva con estado fallido; la siguiente finalizó después de la corrección de reintentos del cliente. No se mezclan muestras de ambas. Con cinco repeticiones el p95 calculado por rango más cercano es el máximo de la muestra; se interpreta como observación de esa corrida y no como caracterización robusta de colas.



10. Escenarios obligatorios de razonamiento

Tabla 9. Respuesta del diseño a los cuatro escenarios

Escenario	Comportamiento y evidencia
Dos procesos escriben el mismo bloque	Solo un lock vigente permite preparar. El segundo recibe 423 o, tras esperar y cambiar la base, 409. Los cambios de bloques distintos pueden fusionarse.
Una réplica se desconecta durante la escritura	La publicación exige dos recibos auténticos y distintos de cada bloque preparado. Si falta la tercera copia se repara; con menos de dos no se publica. Lectores conservan su versión.
Un nodo falla tras recibir parte del bloque	El temporal incompleto no obtiene recibo ni se publica. La regresión envía un stream incompleto, comprueba DATA_LOSS y rechazo del cierre.
El cliente desconoce las ubicaciones físicas	Consulta coordinadores por nombres lógicos; open/manifest devuelve ubicaciones y capacidades. Ante fallo refresca el mismo snapshot y prueba réplicas autorizadas.

11. Contribuciones, limitaciones y cierre

Tabla 10. Integración y revisión identificables en el repositorio

Integrante	Ámbito registrado	Commits representativos
Samuel Parra Cadavid	Cliente, CLI, regresiones y documentación de uso	3116445; 9d41958
Juan José García Duque	Nodo de datos, contrato gRPC, identidades, clúster local y guiones	558a8d0; c503e80; 32fdf44
Andrés Felipe Martínez Velasquez	Control y catálogo, despliegue, correcciones del cliente, pruebas AWS e informe	e20d902; 3da88d5; 87836a9; 6f765cc; b076042

Los commits identifican integración y revisión de componentes; cada integrante debe poder explicar la arquitectura completa y las pruebas asociadas. El repositorio del equipo está disponible en https://github.com/samuel2524/Proyecto-Nube-Dfsha.

Quedan como límites y trabajo experimental adicional la comparación 4/8 MiB y 3/4 nodos, instrumentar CPU/memoria/disco/NIC, estudiar el epoch global bajo mayor carga y probar la pérdida completa de una VM. El catálogo necesita políticas operativas de respaldo y retención; no hay migración automática del prototipo anterior ni cuotas por usuario.

La comparación de propuestas y los cambios de alcance se describen en las secciones 2 y 3 de este informe. Las limitaciones de RF3 y de las garantías no funcionales se declaran para su revisión conforme al alcance asignado. El video final de 10 a 15 minutos y la defensa individual son entregables complementarios que deben mostrar ejecución distribuida, fallos, resultados y explicación de decisiones.

Referencias

[1] S. Ghemawat, H. Gobioff y S.-T. Leung, "The Google file system," en Proc. 19th ACM Symp. Operating Systems Principles, 2003, pp. 29-43.

[2] K. Shvachko, H. Kuang, S. Radia y R. Chansler, "The Hadoop distributed file system," en Proc. IEEE MSST, 2010, pp. 1-10.

[3] etcd Authors, "etcd documentation," 2026. Disponible: https://etcd.io/docs/

[4] A. Ospina Sanjuan, "gRPC: de la invocación local a los microservicios modernos," diapositivas de clase, Universidad Pontificia Bolivariana, 2026.

[5] J. H. Howard et al., "Scale and performance in a distributed file system," ACM Trans. Computer Systems, vol. 6, no. 1, pp. 51-81, 1988.

[6] M. Jones, J. Bradley y N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015.

[7] E. Rescorla, "The Transport Layer Security (TLS) protocol version 1.3," RFC 8446, IETF, 2018.

[8] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST SP 800-38D, 2007.

[9] D. Ongaro y J. Ousterhout, "In search of an understandable consensus algorithm," en Proc. USENIX ATC, 2014, pp. 305-319.

[10] E. Brewer, "CAP twelve years later: How the 'rules' have changed," Computer, vol. 45, no. 2, pp. 23-29, 2012.

[11] M. van Steen y A. S. Tanenbaum, Distributed Systems, 4.ª ed. distributed-systems.net, 2023.